Companion
to

ChatGPT Teaches TikZ

Chapter 18: Reusable Styles for Large Drawings (I)

Contents

18.1 What This Companion Adds

Chapter 18 introduced named styles as a remedy for repeated option lists. This Companion develops a practical refactoring method: detect repetition, identify the graphical role, define one semantic style, and verify that the geometry has not changed.

Design

Coordinates and paths describe the figure. Styles describe its appearance. A refactoring should change the source organization without changing the picture.

18.2 A Style Vocabulary

Form	Purpose
`name/.style={...}`	Defines a reusable collection of options.
`\tikzset{...}`	Defines styles outside an option list.
`vertex`	A semantic name for a graph vertex style.
`edge`	A semantic name for a graph edge style.
`construction`	A semantic name for an auxiliary-line style.
`label`	A semantic name for explanatory text.
`style A,style B`	Applies two independent styles to one object.
`scope`	Limits a temporary definition or modification.

18.3 Recognizing Repetition

This picture repeats the same vertex options three times.

```
\node[draw,circle,fill=blue!15,minimum size=8mm]
  (A) at (0,0) {$A$};
\node[draw,circle,fill=blue!15,minimum size=8mm]
  (B) at (2,1) {$B$};
\node[draw,circle,fill=blue!15,minimum size=8mm]
  (C) at (2,-1) {$C$};
```

The repetition is not merely long. It creates several places where a later design change can become inconsistent.

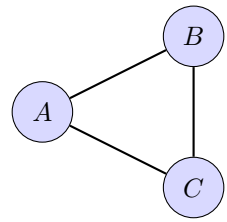
18.4 Defining the First Styles

```
\tikzset{
  vertex/.style={
    draw,circle,fill=blue!15,minimum size=8mm
  },
  edge/.style={thick}
}
```

The drawing commands now state roles.

```
\node[vertex] (A) at (0,0) {$A$};
\node[vertex] (B) at (2,1) {$B$};
\node[vertex] (C) at (2,-1) {$C$};
\draw[edge] (A)--(B)--(C)--(A);
```

See



18.5 Local and Document-Level Styles

A style in the `tikzpicture` option list belongs only to that picture.

```
\begin{tikzpicture}[
  vertex/.style={draw,circle,minimum size=8mm}
]
  ...
\end{tikzpicture}
```

A definition with `\tikzset` may be placed before several pictures so that they share one design.

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  edge/.style={thick}
}
```

Choose the smallest scope that matches the intended reuse.

18.6 Naming by Meaning

Names such as `blue circle` and `thick line` describe the current appearance. Names such as `vertex`, `edge`, and `construction` describe roles that survive a redesign.

Common Mistake

A style named `blue vertex` becomes misleading if the publisher later requests gray figures. Prefer `ordinary vertex` or another semantic name.

18.7 Combining Independent Roles

An object can use more than one style.

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  selected/.style={very thick,fill=yellow!25}
}
\node[vertex,selected] at (0,0) {$B$};
```

The first style says what the object is. The second records its special role in this particular figure.

18.8 Testing a Refactoring

Refactor in three passes:

1. copy each repeated option list into a named style;
2. replace the lists with style names; and
3. compile before changing any visual value.

Only after the old and refactored pictures agree should you redesign the style.

18.9 One Edit, Many Objects

Changing

```
vertex/.style={draw,circle,fill=blue!15,minimum size=8mm}
```

to

```
vertex/.style={draw,rounded corners,fill=gray!15,
               minimum width=10mm,minimum height=7mm}
```

updates every object that uses `vertex`. No coordinate or edge changes.

18.10 Worked Project: Refactoring a Process Diagram

18.10.1 Stage 1: Define the Visual Roles

```
\tikzset{
  process/.style={
    draw,rounded corners,fill=blue!10,
    minimum width=19mm,minimum height=8mm
  },
  flow/.style={thick},
  note/.style={font=\small,fill=white}
}
```

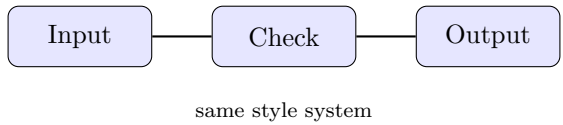
18.10.2 Stage 2: Describe Only the Structure

```
\node[process] (A) at (0,0) {Input};
\node[process] (B) at (2.7,0) {Check};
\node[process] (C) at (5.4,0) {Output};
\draw[flow] (A)--(B)--(C);
\node[note] at (2.7,-1) {same style system};
```

18.10.3 The Complete Picture

```
\begin{tikzpicture}[
  process/.style={
    draw,rounded corners,fill=blue!10,
    minimum width=19mm,minimum height=8mm
  },
  flow/.style={thick},
  note/.style={font=\small,fill=white}
]
  \node[process] (A) at (0,0) {Input};
  \node[process] (B) at (2.7,0) {Check};
  \node[process] (C) at (5.4,0) {Output};
  \draw[flow] (A)--(B)--(C);
  \node[note] at (2.7,-1) {same style system};
\end{tikzpicture}
```

See



same style system

Design

The drawing commands contain structure and text. All repeated presentation decisions belong to the style definitions.

18.11 Debugging Workshop

18.11.1 A Style Has No Effect

Check its spelling and confirm that the definition appears before the object that uses it.

18.11.2 A Refactoring Changes the Picture

Compare the original option list with the style definition. An option was omitted, added, or applied in a different order.

18.11.3 The Style Name Becomes False

Rename visual styles by role before redesigning their appearance.

18.11.4 One Exception Creates a New Style

For a single object, combine the base style with one local option. Create a new style only when the exception represents a recurring role.

18.12 Exercises

1. Replace three repeated node option lists with one style.
2. Define separate styles for vertices and edges.
3. Define an auxiliary construction-line style.
4. Rename two appearance-based styles semantically.

5. Combine a base style with a selected-object style.
6. Move a picture-local style to `\tikzset`.
7. Redesign every vertex by editing one definition.
8. Refactor an earlier diagram without changing its appearance.

18.13 Solutions

18.13.1 Exercises 1–4

```
\tikzset{
  vertex/.style={draw,circle,minimum size=8mm},
  edge/.style={thick},
  construction/.style={densely dashed,gray}
}
```

For Exercise 4, replace names that mention color, shape, or thickness with names that identify the object's role.

18.13.2 Exercises 5–7

```
\node[vertex,selected] at (0,0) {$A$};
```

Exercise 6 moves the unchanged style definition outside the picture. Exercise 7 changes only that definition.

For Exercise 8, compile before and after refactoring and compare the rendered figures before making design changes.

18.14 ChatGPT Workshop

Ask ChatGPT to preserve geometry while reorganizing presentation choices.

1. Find every repeated TikZ option list and propose semantic style names.
2. Refactor this picture with styles without changing its appearance.
3. Identify style names that describe appearance rather than meaning.
4. Decide whether this exception needs a new style or one local option.
5. Verify that no coordinate, path, or label changed during refactoring.

18.15 Final Checklist

Before continuing to Chapter 19, check that you can

1. recognize repeated option lists;
2. define and apply a reusable style;
3. choose semantic rather than visual names;
4. combine independent styles on one object;
5. choose an appropriate scope for a definition;
6. verify that refactoring preserves the picture; and
7. redesign many objects through one style edit.